FILE

A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

What is File System?
A file is a collection of correlated information which is recorded on secondary or non-volatile storage like magnetic disks, optical disks, and tapes. It is a method of data collection that is used as a medium for giving input and receiving output from that program.

In general, a file is a sequence of bits, bytes, or records whose meaning is defined by the file creator and user. Every File has a logical location where they are located for storage and retrieval.

Objective of File management System
Here are the main objectives of the file management system:

- It provides I/O support for a variety of storage device types.
- Minimizes the chances of lost or destroyed data
- Helps OS to standardized I/O interface routines for user processes.
- It provides I/O support for multiple users in a multiuser systems environment.

Properties of a Filesystem

- Files are stored on a storage medium such as disk and do not vanish when a user logs out of the computer system.
- With each file are associated access permissions, which permit controlled sharing of that file.
- Files may form arranged or complex structures according to the relationship among them.
- Several files can be grouped together under a directory.
- A directory also referred to as a folder also has attributes similar to those of a file, such as a name, size, location, access permissions, etc.
- A file system also provides several features such as a crash recovery mechanism, data loss/corruption prevention, etc.

File Attributes

A file has a name and data. Moreover, it also stores meta information like file creation date and time, current size, last modified date, etc. All this information is called the attributes of a file system.

Here, are some important File attributes used in OS:

- **Name:** It is the only information stored in a human-readable form.
- **Identifier**: Every file is identified by a unique tag number within a file system known as an identifier.
- **Location:** Points to file location on device.
- **Type:** This attribute is required for systems that support various types of files.
- **Size**. Attribute used to display the current file size.
- **Protection**. This attribute assigns and controls the access rights of reading, writing, and executing the file.
- **Time, date and security:** It is used for protection, security, and also used for monitoring

Different types of files are:

- Executable file
  In an executable file, the binary code that is loaded in the memory for execution is stored. It is stored in an exe type file.

- Source file
  The source file has subroutines and functions that are compiled later.

- Object file
  An object file is a sequence of bytes used by the linker.

- Text file
  A text file is a sequence of characters.

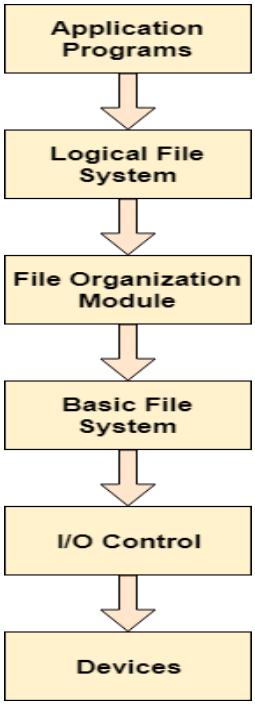
Functions of File

- Create file, find space on disk, and make an entry in the directory.
- Write to file, requires positioning within the file
- Read from file involves positioning within the file
- Delete directory entry, regain disk space.
- Reposition: move read/write position.

FILE SYSTEM IMPLEMENTATION IN OPERATING SYSTEM

File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file.

file system is divided in different layers, and also the functionality of each layer.



- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.
- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.
- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.

- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

Advantages :

1. Duplication of code is minimized.
2. Each file system can have its own logical file system.

Disadvantages :
If we access many files at same time then it results in low performance.

We can use the following data structures to implement file systems:

1. On-disk structures
2. In-memory structures

On-disk structures

They store information about the total number of disk blocks, their location, free disk blocks, etc. Different on-disk structures are as follows:

- **Directory structure**

  Directory structure stores the file names and associated inode numbers.

- **Volume Control Block**

  Volume Control Block stores information about a particular partition. Examples: block pointers, free block count, etc.

- **Boost Control Block**

  Boost Control Block is generally the first block of volume and it stores the information needed to boot an operating system. It is also known as partition boot sector in NTFS and boot block in UNIX.

- **Per-File FCB**

  Per-File FCB stores details about files and has a unique identifier number to associate with the directory entry.

In-memory structures
The in-memory structure is maintained in the main memory and is helpful for file management for caching. Different in-memory structures are:

- **Mount Table**

  Mount Table stores information about the mounted volume.

- **Per-process open-file table**

  Per-process open-file table stores information opened by the process and it maps with the system open-file.

- **Directory-structure cache**

  Directory-structure cache stores information about the recently accessed directories.

- **System-wide open-file table**

  The system-wide open-file table stores the copy of the FCB of each open file.

SECURITY PROTECTION

Protection and security requires that computer resources such as CPU, softwares, memory etc. are protected. This extends to the operating system as well as the data in the system. This can be done by ensuring integrity, confidentiality and availability in the operating system. The system must be protect against unauthorized access, viruses, worms etc.

Threats to Protection and Security

A threat is a program that is malicious in nature and leads to harmful effects for the system. Some of the common threats that occur in a system are −

Virus

Viruses are generally small snippets of code embedded in a system. They are very dangerous and can corrupt files, destroy data, crash systems etc. They can also spread further by replicating themselves as required.

Trojan Horse

A trojan horse can secretly access the login details of a system. Then a malicious user can use these to enter the system as a harmless being and wreak havoc.

Trap Door

A trap door is a security breach that may be present in a system without the knowledge of the users. It can be exploited to harm the data or files in a system by malicious people.

Worm

A worm can destroy a system by using its resources to extreme levels. It can generate multiple copies which claim all the resources and don't allow any other processes to access them. A worm can shut down a whole network in this way.

Denial of Service

These type of attacks do not allow the legitimate users to access a system. It overwhelms the system with requests so it is overwhelmed and cannot work properly for other user.

Protection and Security Methods

The different methods that may provide protect and security for different computer systems are −

Authentication

This deals with identifying each user in the system and making sure they are who they claim to be. The operating system makes sure that all the users are authenticated before they access the system. The different ways to make sure that the users are authentic are:

- **Username/ Password**

Each user has a distinct username and password combination and they need to enter it correctly before they can access the system.

- **User Key/ User Card**

  The users need to punch a card into the card slot or use they individual key on a keypad to access the system.

- **User Attribute Identification**

  Different user attribute identifications that can be used are fingerprint, eye retina etc. These are unique for each user and are compared with the existing samples in the database. The user can only access the system if there is a match.

One Time Password

These passwords provide a lot of security for authentication purposes. A one time password can be generated exclusively for a login every time a user wants to enter the system. It cannot be used more than once. The various ways a one time password can be implemented are −

- **Random Numbers**

  The system can ask for numbers that correspond to alphabets that are pre arranged. This combination can be changed each time a login is required.

- **Secret Key**

  A hardware device can create a secret key related to the user id for login. This key can change each time.

I/O Hardware in Operating System

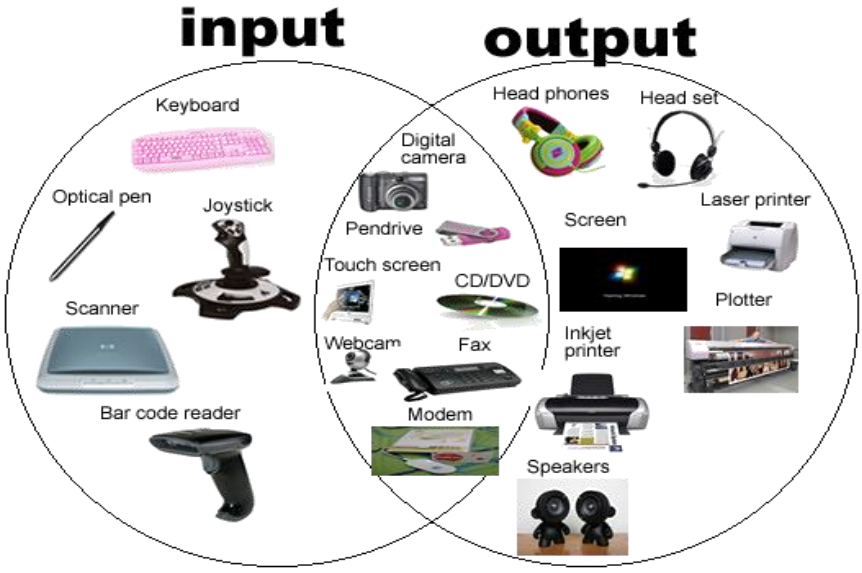
I/O Hardware is a set of specialized hardware devices that help the operating system access disk drives, printers, and other peripherals. These devices are located inside the motherboard and connected to the processor using a bus. They often have specialized controllers that allow them to quickly respond to requests from software running on top of them or even respond directly to commands from an application program.

I/O Hardware in Operating System

I/O Hardware is a set of specialized hardware devices that help the operating system access disk drives, printers, and other peripherals. These devices are located inside the motherboard and connected to the processor using a bus. They often have specialized controllers that allow them to quickly respond to requests from software running on top of them or even respond directly to commands from an application program.

I/O Hardware in Operating System

I/O Hardware is a set of specialized hardware devices that help the operating system access disk drives, printers, and other peripherals. These devices are located inside the motherboard and connected to the processor using a bus. They often have specialized controllers that allow them to quickly respond to requests from software running on top of them or even respond directly to commands from an application program.



I/O Devices

I/O devices can be roughly divided into two categories: block devices and character devices.

- **Block Devices**
  It stores information in fixed-size blocks, each one with its own address.
  Common block sizes range from 512 to 65,536 bytes.
  Examples are hard disks, Blu-ray discs, and USB sticks.

- **Character Device**
  It delivers or accepts a stream of characters, without regard to any block structure. Not addressable and does not have any seek operation.
  Examples are printers, network interfaces, mice, and most other devices that are not disk-like can be seen as character devices.

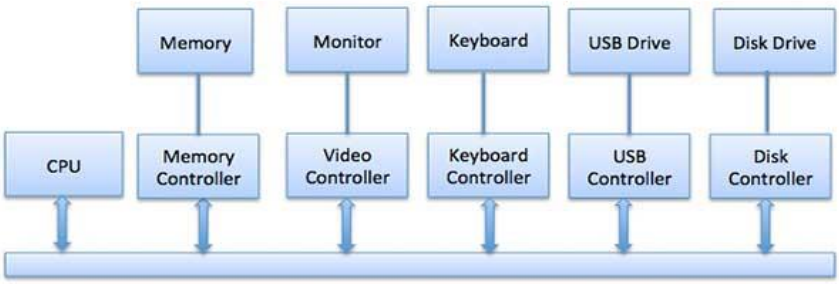
Device Controllers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices.

The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.

There is always a device controller and a device driver for each device to communicate with the Operating Systems. A device controller may be able to handle multiple devices. As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.

Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller. Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.



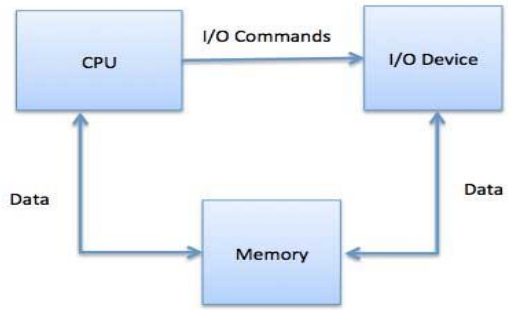
Synchronous vs asynchronous I/O

- **Synchronous I/O** − In this scheme CPU execution waits while I/O proceeds
- **Asynchronous I/O** − I/O proceeds concurrently with CPU execution

COMMUNICATION TO I/O DEVICES

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

- Special Instruction I/O
- Memory-mapped I/O
- Direct memory access (DMA)



While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU. I/O device operates asynchronously with CPU, interrupts CPU when finished.

The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device. Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

DIRECT MEMORY ACCESS (DMA)

Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead.

Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

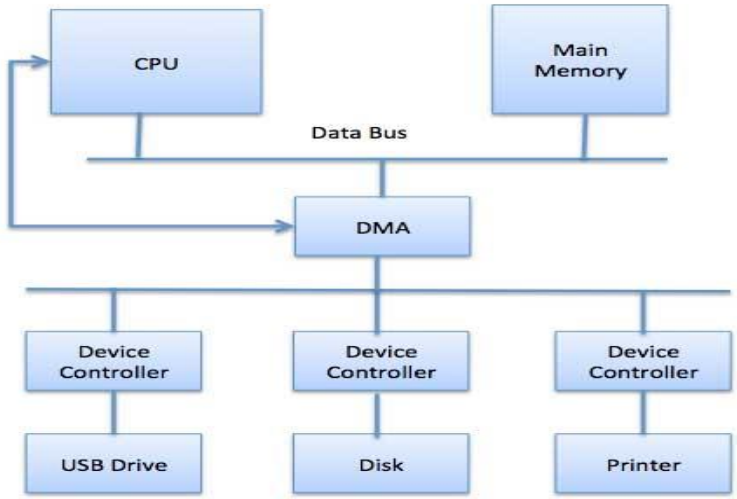
Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

Advantages:

1. Transferring the data without the involvement of the processor will **speed up** the read-write task.
2. DMA **reduces the clock cycle** requires to read or write a block of data.
3. Implementing DMA also **reduces the overhead** of the processor.

Disadvantages

1. As it is a hardware unit, it would **cost** to implement a DMA controller in the system.
2. Cache **coherence** problem can occur while using DMA controller.



The operating system uses the DMA hardware as follows −

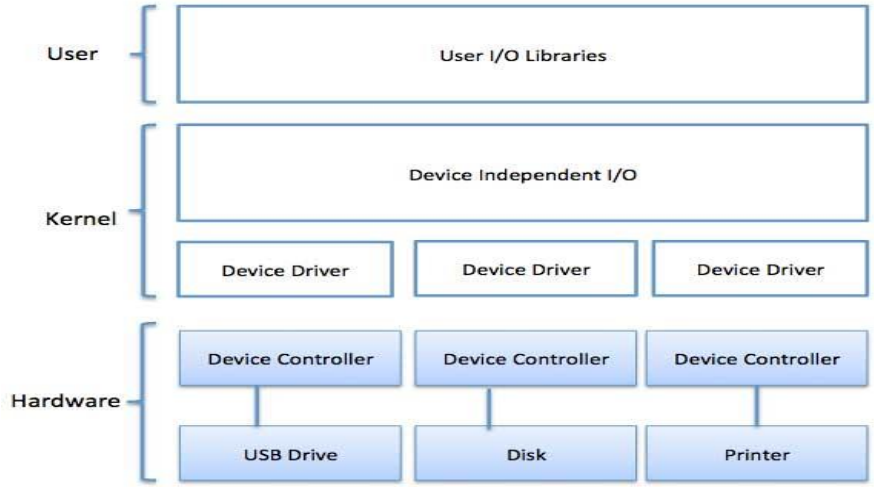
Step	Description
1	Device driver is instructed to transfer disk data to a buffer address X.
2	Device driver then instruct disk controller to transfer data to buffer.
3	Disk controller starts DMA transfer.
4	Disk controller sends each byte to DMA controller.
5	DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.
6	When C becomes zero, DMA interrupts CPU to signal transfer completion.

PRINCIPLES OF I/O SOFTWARE

I/O software is often organized in the following layers −

- **User Level Libraries** − This provides simple interface to the user program to perform input and output. For example, **stdio** is a library provided by C and C++ programming languages.
- **Kernel Level Modules** − This provides device driver to interact with the device controller and device independent I/O modules used by the device drivers.
- **Hardware** − This layer includes actual hardware and hardware controller which interact with the device drivers and makes hardware alive.

A key concept in the design of I/O software is that it should be device independent where it should be possible to write programs that can access any I/O device without having to specify the device in advance. For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.



Device Drivers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices. Device drivers encapsulate device-dependent code and implement a standard interface in such a way that code contains device-specific register reads/writes. Device driver, is generally written by the device's manufacturer and delivered along with the device on a CD-ROM.

A device driver performs the following jobs −

- To accept request from the device independent software above to it.
- Interact with the device controller to take and give I/O and perform required error handling
- Making sure that the request is executed successfully

How a device driver handles a request is as follows: Suppose a request comes to read a block N. If the driver is idle at the time a request arrives, it starts carrying out the request immediately. Otherwise, if the driver is already busy with some other request, it places the new request in the queue of pending requests.

Interrupt handlers

An interrupt handler, also known as an interrupt service routine or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt.

When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.

The interrupt mechanism accepts an address ─ a number that selects a specific interrupt handling routine/function from a small set. In most architectures, this address is an offset stored in a table called the interrupt vector table. This vector contains the memory addresses of specialized interrupt handlers.

Device-Independent I/O Software

The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software. Though it is difficult to write completely device independent software but we can write some modules which are common among all the devices. Following is a list of functions of device-independent I/O Software −

- Uniform interfacing for device drivers
- Device naming - Mnemonic names mapped to Major and Minor device numbers
- Device protection
- Providing a device-independent block size
- Buffering because data coming off a device cannot be stored in final destination.
- Storage allocation on block devices
- Allocation and releasing dedicated devices
- Error Reporting

User-Space I/O Software

These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers. Most of the user-level I/O software consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system.

I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O SW. For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming.

Kernel I/O Subsystem

Kernel I/O Subsystem is responsible to provide many services related to I/O. Following are some of the services provided.

- **Scheduling** − Kernel schedules a set of I/O requests to determine a good order in which to execute them. When an application issues a blocking I/O system call, the request is placed on the queue for that device. The Kernel I/O scheduler rearranges the order of the queue to improve the overall system efficiency and the average response time experienced by the applications.
- **Buffering** − Kernel I/O Subsystem maintains a memory area known as **buffer** that stores data while they are transferred between two devices or between a device with an application operation. Buffering is done to cope with a speed mismatch between the producer and consumer of a data stream or to adapt between devices that have different data transfer sizes.
- **Caching** − Kernel maintains cache memory which is region of fast memory that holds copies of data. Access to the cached copy is more efficient than access to the original.
- **Spooling and Device Reservation** − A spool is a buffer that holds output for a device, such as a printer, that cannot accept interleaved data streams. The spooling system copies the queued spool files to the printer one at a time. In some operating systems, spooling is managed by a system daemon process. In other operating systems, it is handled by an in kernel thread.
- **Error Handling** − An operating system that uses protected memory can guard against many kinds of hardware and application errors.

DISK HARDWARE

- Magnetic disks
- RAID
- CD-ROM
- CD-Recordable
- CD-Rewritable
- DVD

Magnetic Disks

Magnetic disks are organised into cylinders where each one containing as many tracks as there are heads stacked vertically. And the tracks are again divided into sectors.

RAID

RAID stands for Redundant Array of Inexpensive Disks.

The basic idea behind Redundant Array of Inexpensive Disks is just to install a box full of disks next to the computer, typically a large server, replace the disk controller card with a RAID controller, copy the data over to the RAID and then continue the normal operation.

In other word, you can say that RAID looks like a SLED (Single Large Expensive Disk) to the OS.

But RAID have better performance and reliability than SLED.

CD-ROM

In the year of 1980, major two company namely Philips and Sony, together developed the Compact Disk (CD) which rapidly replaced 33 1/3-rpm vinyl record for music.

All the compact disks are 120mm across and 1.2mm thick, with a 15mm hole in the middle of the CD.

Now, after almost 4 years, that is, in the year of 1984, these two company now realised the power and potential of using the compact disks to store the computer data, therefore they developed CD-ROM, that stands for Compact Disk-Read Only Memory.

CD-Recordable

Physically, CD-Recordable (CD-R) stars with 120mm polycarbonate blanks that are like CD-ROMs, except that they contain a 0.6mm wide groove to guide the laser for writing.

CD-Rewritable

After the people's demand for a rewritable CD-ROM, a technology was developed named CD-ReWritable (CD-RW), that uses the same size media as CD-Recordable (CD-R).

But, CD-RW uses an alloy of silver, antimony, indium, and tellurium for the recording layer.

DVD

DVD stands for Digital Video Disk or Digital Versatile Disk.

DVD uses the same general design as of CD with 120mm injection-moulded polycarbonate disks containing pits and lands that are illuminated by a laser diode and read by a photo detector.

DISK SCHEDULING ALGORITHM

Disk Scheduling Algorithm is an algorithm that keeps and manages input and output requests arriving for the disk in a system. As we know, for executing any process memory is required. And when it comes to accessing things from a hard disk, the process becomes very slow as a hard disk is the slowest part of our computer. There are various methods by which the process can be scheduled and can be done efficiently.

- The algorithms used for disk scheduling are called as **disk scheduling algorithms**.
- The purpose of disk scheduling algorithms is to reduce the total seek time.

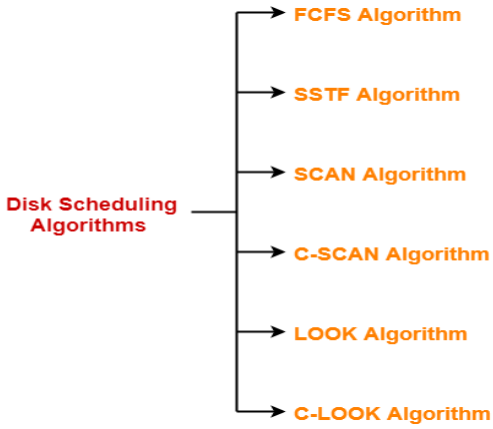
Why is Disk Scheduling needed?

In our system, **multiple requests** are coming to the disk simultaneously which will make a queue of requests. This queue of requests will result in an increased waiting time of requests. The requests wait until the underprocessing request executes. To overcome this queuing and manage the timing of these requests, 'Disk Scheduling' is important in our Operating System.

Important terms related to disk scheduling

There are many terms that we need to know for a better understanding of Disk Scheduling. We are going to have a brief look at each of them one by one:

1. **Seek Time**: As we know, the data may be stored on various blocks of disk. To access these data according to the request, the disk arm moves and find the required block. The time taken by the arm in doing this search is known as "Seek Time".
2. **Rotational Latency**: The required data block needs to move at a particular position from where the read/write head can fetch the data. So, the time taken in this movement is known as "Rotational Latency". This rotational time should be as less as possible so, the algorithm that will take less time to rotate will be considered a better algorithm.
3. **Transfer Time**: When a request is made from the user side, it takes some time to fetch these data and provide them as output. This taken time is known as "Transfer Time".
4. **Disk Access Time**: It is defined as the total time taken by all the above processes. Disk access time = (seek time + rotational latency time + transfer time)
5. **Disk Response Time**: The disk processes one request at a single time. So, the other requests wait in a queue to finish the ongoing process of request. The average of this waiting time is called "Disk Response Time".
6. **Starvation**: Starvation is defined as the situation in which a low-priority job keeps waiting for a long time to be executed. The system keeps sending high-priority jobs to the disk scheduler to execute first.



1. FCFS disk scheduling algorithm-

It stands for 'first-come-first-serve'. As the name suggests, the request which comes first will be processed first and so on. The requests coming to the disk are arranged in a proper sequence as they arrive. Since every request is processed in this algorithm, so there is no chance of 'starvation'.

- As the name suggests, this algorithm entertains requests in the order they arrive in the disk queue.
- It is the simplest disk scheduling algorithm.

Advantages-

- It is simple, easy to understand and implement.
- It does not cause starvation to any request.

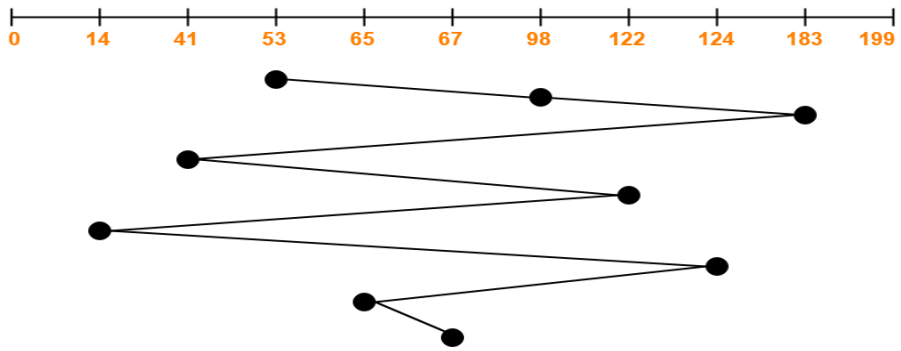
Disadvantages-

- It results in increased total seek time.
- It is inefficient.

Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The FCFS scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests
= (98 – 53) + (183 – 98) + (183 – 41) + (122 – 41) + (122 – 14) + (124 – 14) + (124 – 65) + (67 – 65)
= 45 + 85 + 142 + 81 + 108 + 110 + 59 + 2
= 632

2. SSTF disk scheduling algorithm-

It stands for 'Shortest seek time first'. As the name suggests, it searches for the request having the least 'seek time' and executes them first. This algorithm has less 'seek time' as compared to FCFS Algorithm.

- SSTF stands for **Shortest Seek Time First**.
- This algorithm services that request next which requires least number of head movements from its current position regardless of the direction.
- It breaks the tie in the direction of head movement.

Advantages-

- It reduces the total seek time as compared to **FCFS**.
- It provides increased throughput.
- It provides less average response time and waiting time.

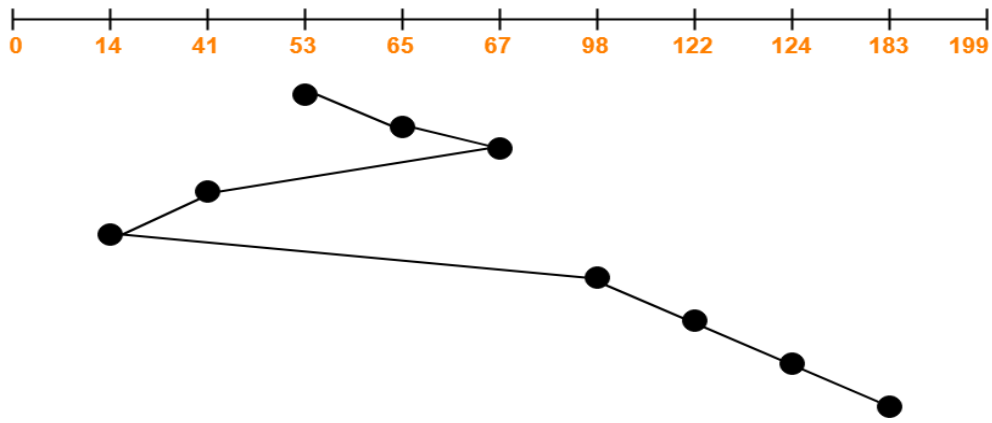
Disadvantages-

- There is an overhead of finding out the closest request.
- The requests which are far from the head might starve for the CPU.
- It provides high variance in response time and waiting time.
- Switching the direction of head frequently slows down the algorithm.

Problem-01:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (67 – 41) + (41 – 14) + (98 – 14) + (122 – 98) + (124 – 122) + (183 – 124)

= 12 + 2 + 26 + 27 + 84 + 24 + 2 + 59

= 236

3. SCAN disk scheduling algorithm:

In this algorithm, the head starts to scan all the requests in a direction and reaches the end of the disk. After that, it reverses its direction and starts to scan again the requests in its path and serves them. Due to this feature, this algorithm is also known as the "Elevator Algorithm".

- As the name suggests, this algorithm scans all the cylinders of the disk back and forth.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction and move towards the starting end servicing all the requests in between.
- The same process repeats.

NOTE-

- SCAN Algorithm is also called as **Elevator Algorithm.**
- This is because its working resembles the working of an elevator.

Advantages-

- It is simple, easy to understand and implement.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

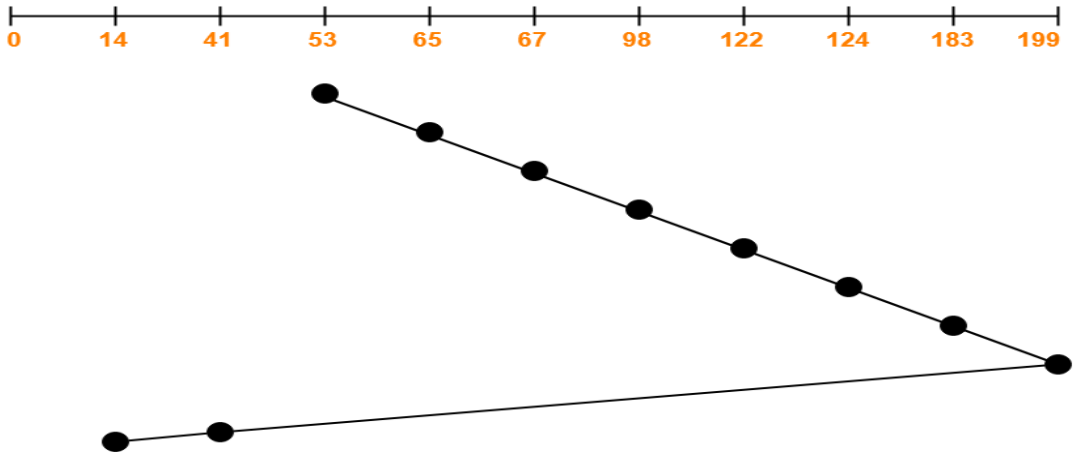
Disadvantages-

- It causes long waiting time for the cylinders just visited by the head.
- It causes the head to move till the end of the disk even if there are no requests to be serviced.

Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests
= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (199 –183) + (199 – 41) + (41 – 14)
= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 158 + 27
= 331

4. C-SCAN disk scheduling algorithm

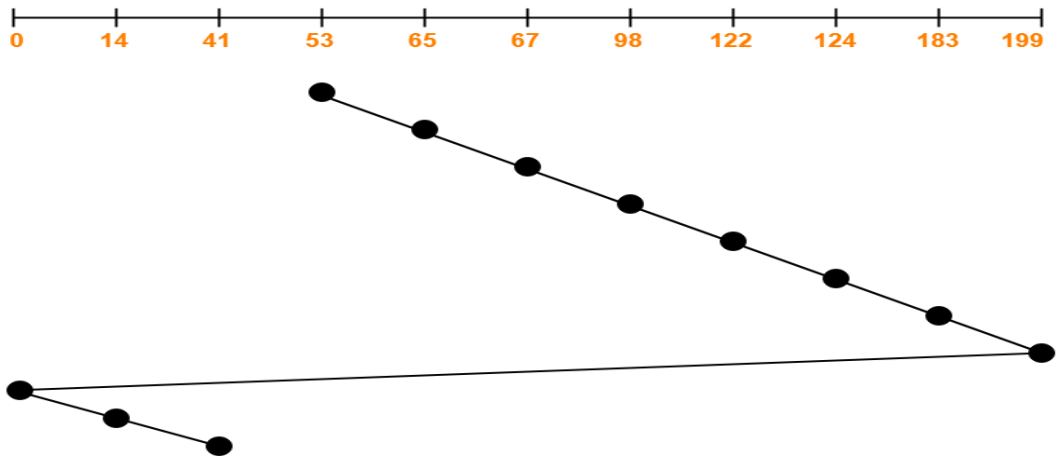
It stands for "Circular-Scan". This algorithm is almost the same as the Scan disk algorithm but one thing that makes it different is that 'after reaching the one end and reversing the head direction, it starts to come back. The disk arm moves toward the end of the disk and serves the requests coming into its path. After reaching the end of the disk it reverses its direction and again starts to move to the other end of the disk but while going back it does not serve any requests.

- Circular-SCAN Algorithm is an improved version of the **SCAN Algorithm**.
- Head starts from one end of the disk and move towards the other end servicing all the requests in between.
- After reaching the other end, head reverses its direction.
- It then returns to the starting end without servicing any request in between.
- The same process repeats.

Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-SCAN scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (199 – 183) + (199 – 0) + (14 – 0) + (41 – 14)

= 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 27

= 386

5. LOOK disk scheduling algorithm:

In this algorithm, the disk arm moves to the 'last request' present and services them. After reaching the last requests, it reverses its direction and again comes back to the starting point. It does not go to the end of the disk, in spite, it goes to the end of requests.

- LOOK Algorithm is an improved version of the **SCAN Algorithm**.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end servicing all the requests in between.
- The same process repeats.

NOTE-

The main difference between SCAN Algorithm and LOOK Algorithm is-

- SCAN Algorithm scans all the cylinders of the disk starting from one end to the other end even if there are no requests at the ends.
- LOOK Algorithm scans all the cylinders of the disk starting from the first request at one end to the last request at the other end.

Advantages-

- It does not causes the head to move till the ends of the disk when there are no requests to be serviced.
- It provides better performance as compared to SCAN Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

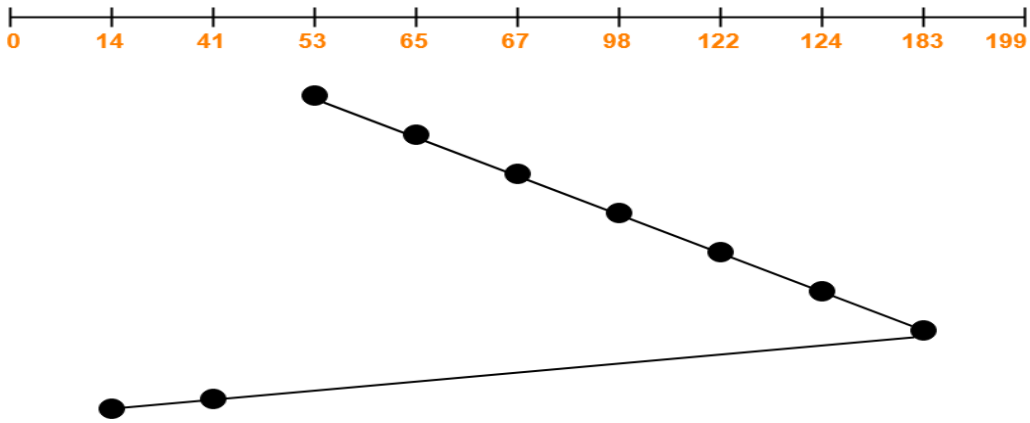
Disadvantages-

- There is an overhead of finding the end requests.
- It causes long waiting time for the cylinders just visited by the head.

Problem-

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests
= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (183 – 41) + (41 – 14)
= 12 + 2 + 31 + 24 + 2 + 59 + 142 + 27
= 299

6. C-LOOK disk scheduling algorithm:

The C-Look algorithm is almost the same as the Look algorithm. The only difference is that after reaching the end requests, it reverses the direction of the head and starts moving to the initial position. But in moving back, it does not serve any requests.

- Circular-LOOK Algorithm is an improved version of the **LOOK Algorithm**.
- Head starts from the first request at one end of the disk and moves towards the last request at the other end servicing all the requests in between.
- After reaching the last request at the other end, head reverses its direction.
- It then returns to the first request at the starting end without servicing any request in between.
- The same process repeats.

Advantages-

- It does not causes the head to move till the ends of the disk when there are no requests to be serviced.
- It reduces the waiting time for the cylinders just visited by the head.
- It provides better performance as compared to LOOK Algorithm.
- It does not lead to starvation.
- It provides low variance in response time and waiting time.

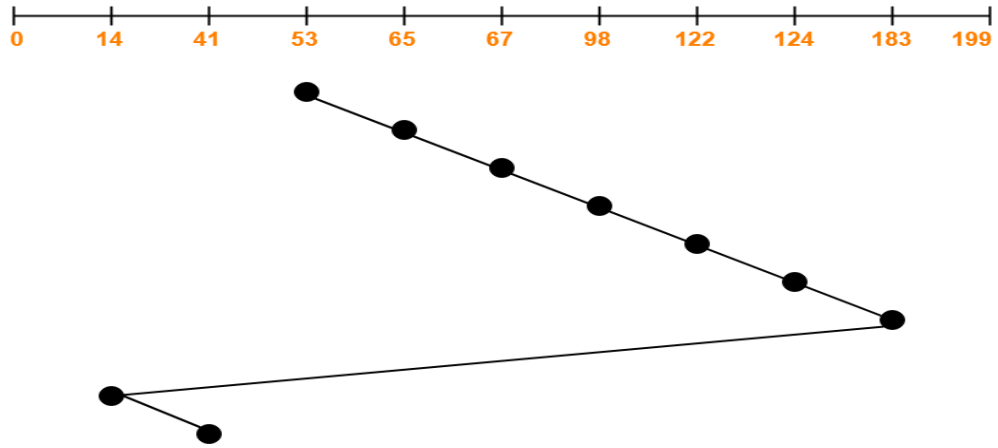
Disadvantages-

- There is an overhead of finding the end requests.

Problem-01:

Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The C-LOOK scheduling algorithm is used. The head is initially at cylinder number 53 moving towards larger cylinder numbers on its servicing pass. The cylinders are numbered from 0 to 199. The total head movement (in number of cylinders) incurred while servicing these requests is _____.

Solution-



Total head movements incurred while servicing these requests

= (65 – 53) + (67 – 65) + (98 – 67) + (122 – 98) + (124 – 122) + (183 – 124) + (183 – 14) + (41 – 14)

= 12 + 2 + 31 + 24 + 2 + 59 + 169 + 27

= 326

What Does Error Handling Mean?

Error handling refers to the response and recovery procedures from error conditions present in a software application. In other words, it is the process comprised of anticipation, detection and resolution of application errors, programming errors or communication errors. Error handling helps in maintaining the normal flow of program execution. In fact, many applications face numerous design challenges when considering error-handling techniques.

Error handling helps in handling both hardware and software errors gracefully and helps execution to resume when interrupted. When it comes to error handling in software, either the programmer develops the necessary codes to handle errors or makes use of software tools to handle the errors. In cases where errors cannot be classified, error handling is usually done with returning special error codes. Special applications known as error handlers are available for certain applications to help in error handling. These applications can anticipate errors, thereby helping in recovering without actual termination of application.

There are four main categories of errors:

- Logical errors
- Generated errors
- Compile-time errors
- Runtime errors

Error-handling techniques for development errors include rigorous proofreading. Error-handling techniques for logic errors or bugs is usually by meticulous application debugging or troubleshooting. Error-handling applications can resolve runtime errors or have their impact minimized by adopting reasonable countermeasures depending on the environment. Most hardware applications include an error-handling mechanism which allows them to recover gracefully from unexpected errors.

As errors could be fatal, error handling is one of the crucial areas for application designers and developers, regardless of the application developed or programming languages used. In worst-case scenarios, the error handling mechanisms force the application to log the user off and shut down the system.

Memory Mapped-2

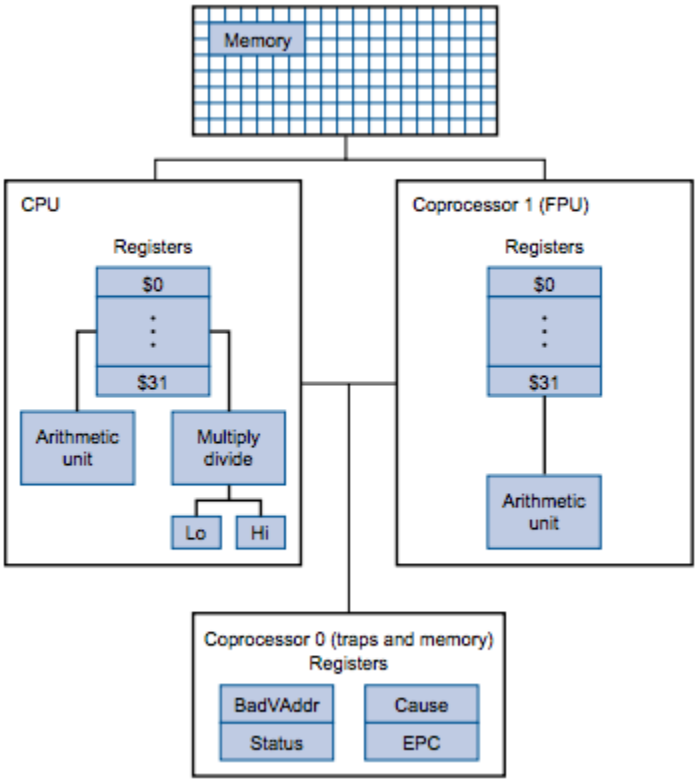
In order to study exceptions and interrupts we will use the Mips (Links to an external site.) 32 simulator Mars (Links to an external site.). All instructions will be written for and tested using Mars but you should be able to use the Spim (Links to an external site.) simulator if you prefer.

Mips system architecture

A Mips processor consists of an integer processing unit (the CPU) and a collection of coprocessors that perform ancillary tasks or operate on other types of data such as floating-point numbers.

Both Mars and Spim simulates two coprocessors:

- **Coprocessor 0** handles exceptions and interrupts.
- **Coprocessor 1** is the floating-point unit.



Integer arithmetic and logical operations are executed directly by the CPU. Floating point operations are executed by Coprocessor 1. Coprocessor 0 are used do manage exceptions and interrupts.

Receiver and transmitter

Mars simulates one I/O device: a memory-mapped console on which a program can read and write characters. The terminal device consists of two independent units: a receiver and a transmitter.

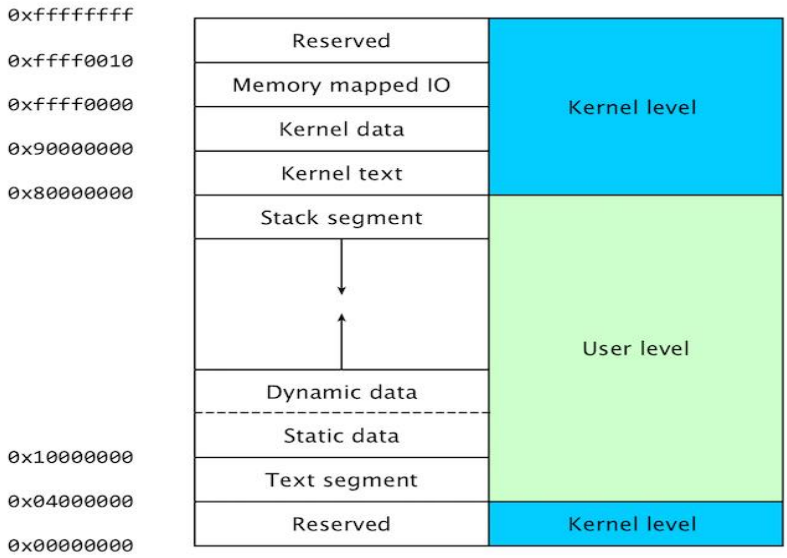
- The **receiver** reads characters typed on the keyboard.
- The **transmitter** display characters on the console.

Echo

The receiver and transmitter are completely independent. This means, for example, that characters typed at the keyboard are not automatically echoed on the display. Instead, a program must explicit echo a character by reading it from the receiver and writing it to the transmitter.

Memory layout

To execute a Mips program memory must be allocated. The Mips computer can address 4 Gbyte of memory, from address 0x0000 0000 to 0xffff ffff. User memory is limited to locations below 0x7fff ffff. In the below figure the layout of the memory allocated to a Mips program is shown.

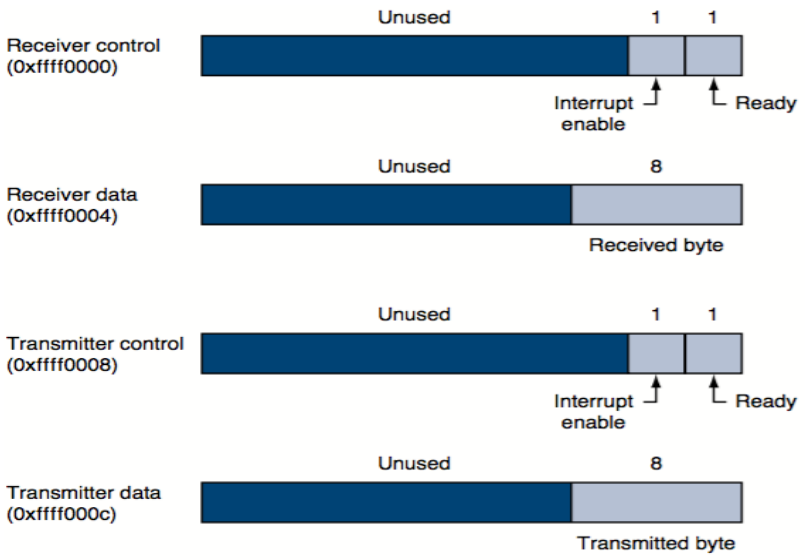


The purpose of the various memory segments:

- The **user level cod**e is stored in the **text segment**.
- **Static data** (data know at compile time) use by the user program is stored in the **data segment**.
- **Dynamic data** (data allocated during runtime) by the user program is stored in the **heap**.
- The **stack** is used by the user program to store temporary data during for example subroutine calls.
- Kernel level code (exception and interrupt handlers) are stored in the **kernel text segment**.
- Static data used by the kernel is stored in the **kernel data segment**.
- Memory mapped registers for IO devices are stored in the **memory mapped IO segment**.

Memory-mapped terminal device

A program controls the terminal with four memory-mapped device registers. Memory-mapped means that each register appears as a special memory location. In this sense, these are not real registers but simply words (32 bit) values stored in memory.



Receiver control

The Receiver Control register is at memory location `0xffff0000`. Only two of its bits are actually used.

Bit 0 is called **ready**:

- If it is1, it means that a character has arrived from the keyboard but has not yet been read from the Receiver Data register.
- The ready bit is read-only: writes to it are ignored.
- The ready bit changes from 0 to 1 when a character is typed at the key- board, and it changes from 1 to 0 when the character is read from the Receiver Data register.

Bit 1 is the keyboard **interrupt enable bit**:

- This bit may be both read and written by a program.
- The interrupt enable is initially 0.
- If it is set to 1 by a program, the terminal requests an interrupt at hardware level 1 whenever a character is typed and the ready bit becomes 1. However, for the interrupt to affect the processor, interrupts must also be enabled in the Status register.
- All other bits of the Receiver Control register are unused.

Receiver data

The second terminal device register is the Receiver Data register at memory address `0xffff0004`.

- The low-order 8 bits of this register contain the last character typed at the keyboard. All other bits contain 0s.
- This register is read-only and changes only when a new character is typed at the keyboard.
- Reading the Receiver Data register resets the ready bit in the Receiver Control register to 0.

- The value in this register is undefined if the Receiver Control register is 0.

Transmitter control

The third terminal device register is the Transmitter Control register at memory address `0xffff0008`. Only the low-order 2 bits of this register are used. They behave much like the corresponding bits of the Receiver Control register.

Bit 0 is called **ready**:

- This bit is read-only.
- If this bit is 1, the transmitter is ready to accept a new character for output.
- If it is 0, the transmitter is still busy writing the previous character.

Bit 1 is the **interrupt enable** bit:

- This bit is readable and writable.
- If this bit is set to 1, then the terminal requests an interrupt at hardware level 0 whenever the transmitter is ready for a new character and the ready bit becomes 1.

Transmitter data

The final device register is the Transmitter Data register at memory address `0xffff000c`.

Only the **least 8 significant bits** (the least significant byte) are used:

- When a value is written into this location, its low-order 8 bits (i.e., an ASCII character) are sent to the console.
- When the Transmitter Data register is written, the ready bit in the Transmitter Control register is reset to 0. This bit stays 0 until enough time has elapsed to transmit the character to the terminal; then the ready bit becomes 1 again.
- The Transmitter Data register should only be written when the ready bit of the Transmitter Control register is 1. If the transmitter is not ready, writes to the Transmitter Data register are ignored (the write appears to succeed but the character is not output).

Timing

In a physical computer sending a character to the terminal doesn’t happen instantly, the operation takes some time to complete.

The simulators Mars and SPIM measures time by counting the number executed instructions , not in real clock time. After writing a character to the receiver data register, this means that the transmitter does not become ready again until the processor executes a fixed number of instructions. If you stop the simulation and look at the ready bit, it will not change. However, if you let the simulation continue, the bit eventually changes back to 1.